

Library Management System

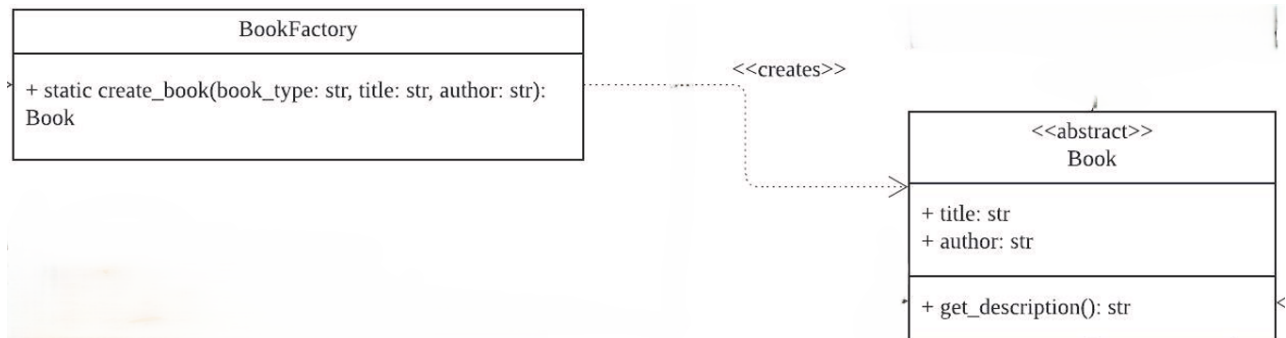
- Fang, Zhou
- Patel, Vivek Thakorbbhai
- Patel, Kevin Harshadkumar

The objective is to design and implement a Library Management System that facilitates the management of books and members within a library. This system should support various functionalities such as adding books, borrowing and returning books, adding members, and searching for books. Additionally, the system should integrate third-party book data and allow for the addition of accessories to books when borrowed. The design should employ multiple software design patterns to ensure scalability, flexibility, and maintainability.

[illegible]

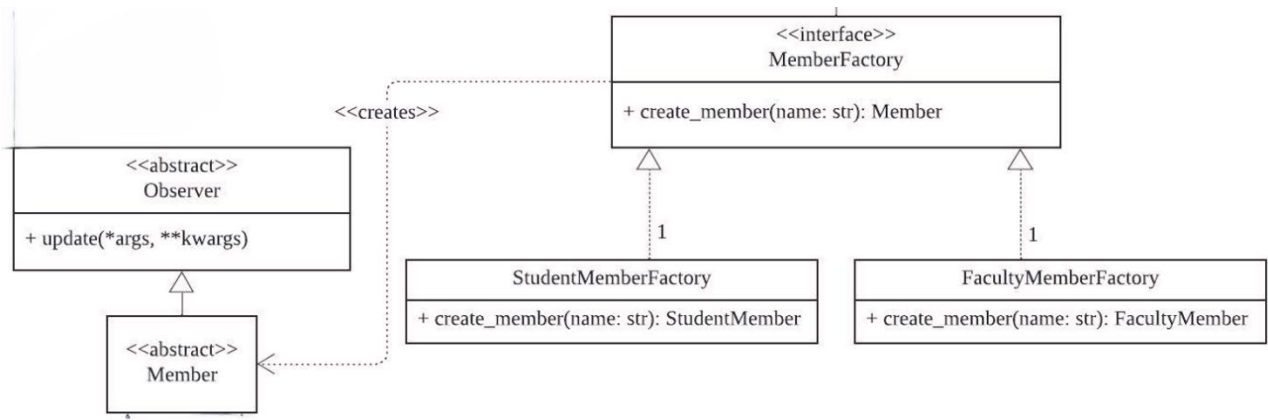
Class Diagram And Design Rational For Each Pattern

Factory Pattern / Factory Method Pattern



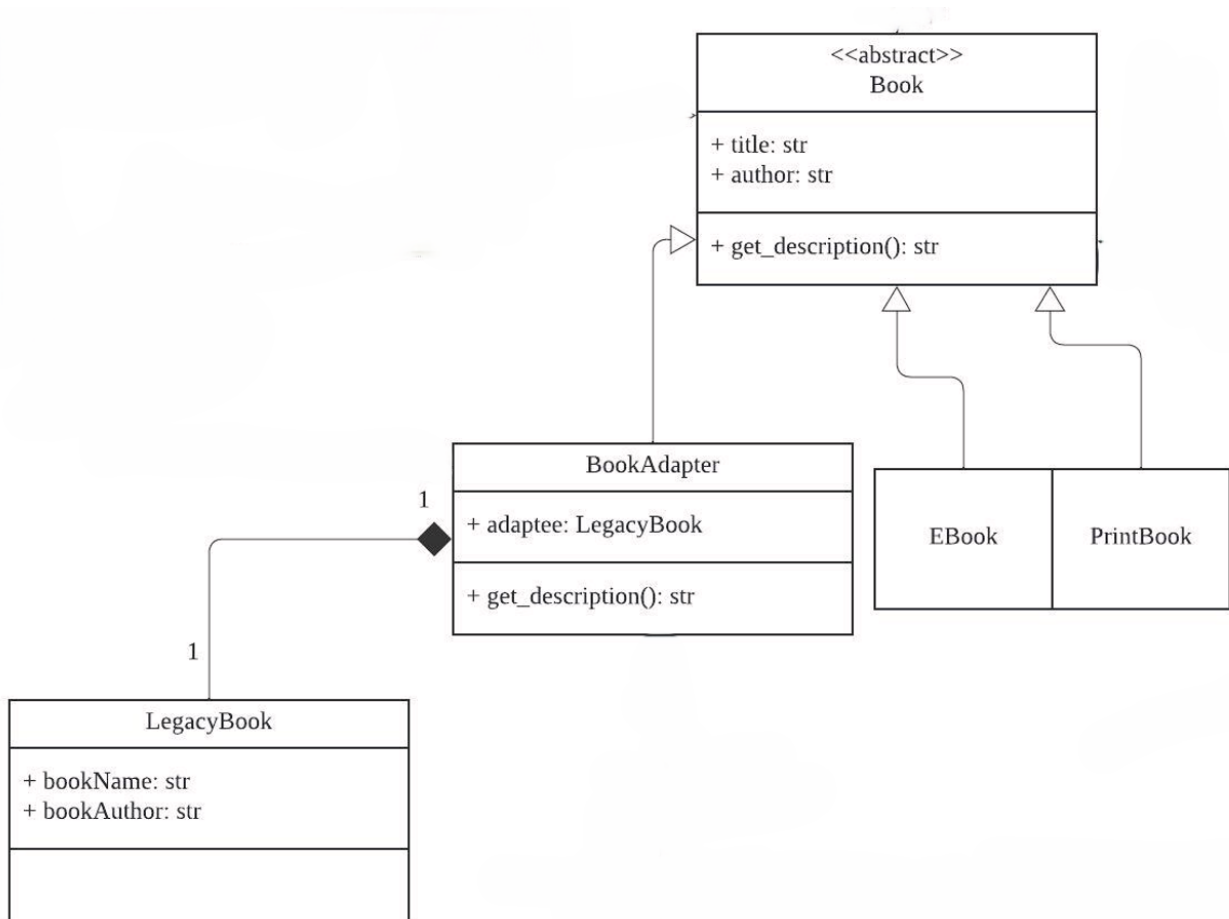
- **Class: BookFactory**
- **Purpose:** Abstract the creation of different types of books (e.g., EBook, PrintBook) to avoid tight coupling between the client code and specific book classes.
- **Implementation:** The BookFactory class has a static method `create_book` that instantiates books based on the specified type.

Abstract Factory Pattern



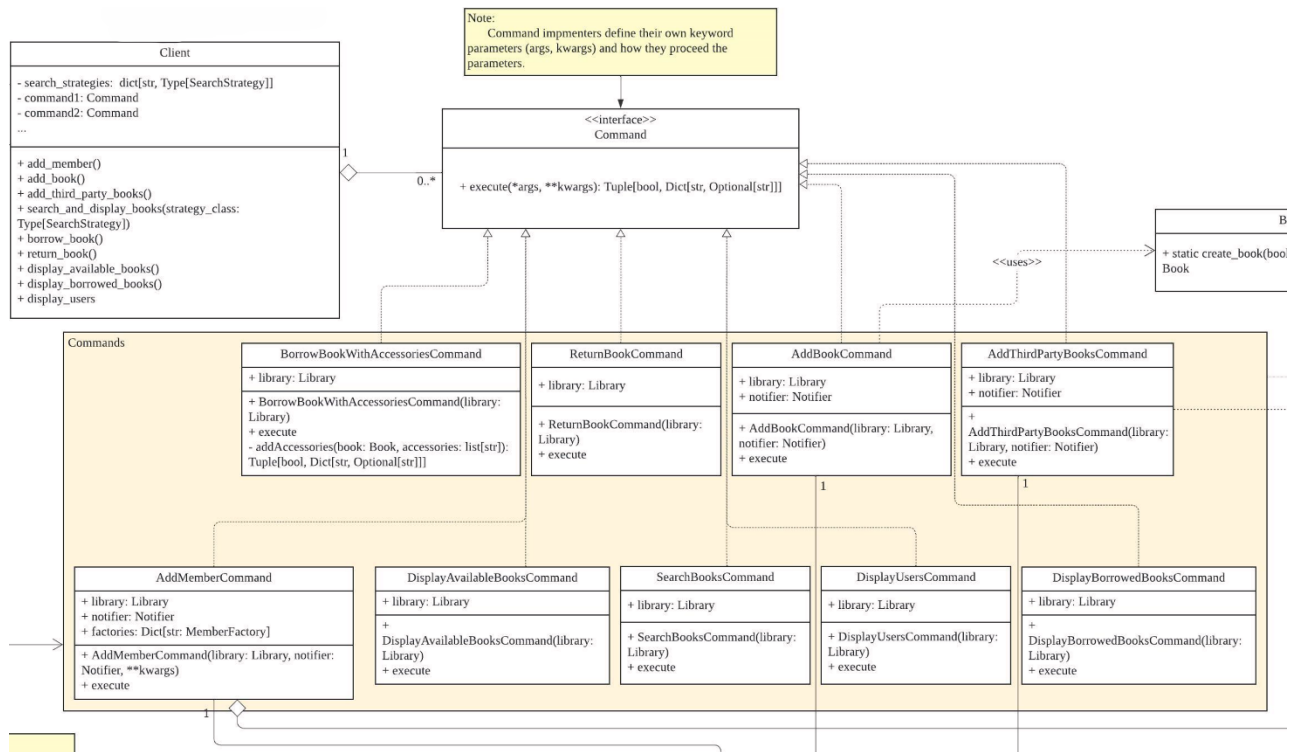
- **Classes:** **MemberFactory**, **StudentMemberFactory**, **FacultyMemberFactory**
- **Purpose:** Provide an interface for creating families of related or dependent objects without specifying their concrete classes.
- **Implementation:** The **MemberFactory** abstract class defines a method `create_member`, which is implemented by **StudentMemberFactory** and **FacultyMemberFactory** to create **StudentMember** and **FacultyMember** objects, respectively.

Adapter Pattern



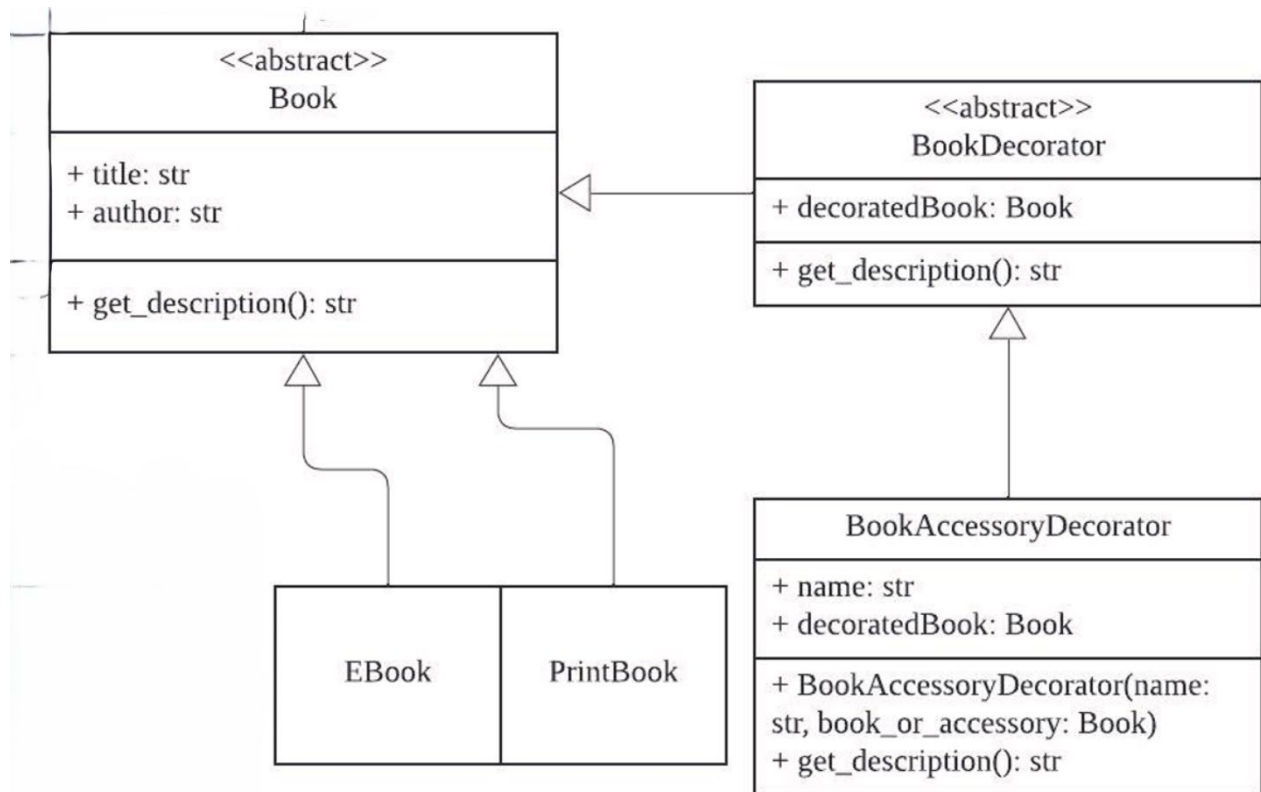
- **Classes:** **ThirdPartyBookAPI**, **LegacyBook**, **BookAdapter**
- **Purpose:** Allow the interface of an existing class to be used as another interface. It is often used to make existing classes work with others without modifying their source code.
- **Implementation:** The **BookAdapter** class adapts the interface of **ThirdPartyBookAPI** to be compatible with the **Book** class used in the library, allowing third-party books to be integrated seamlessly into the system.

Command Pattern



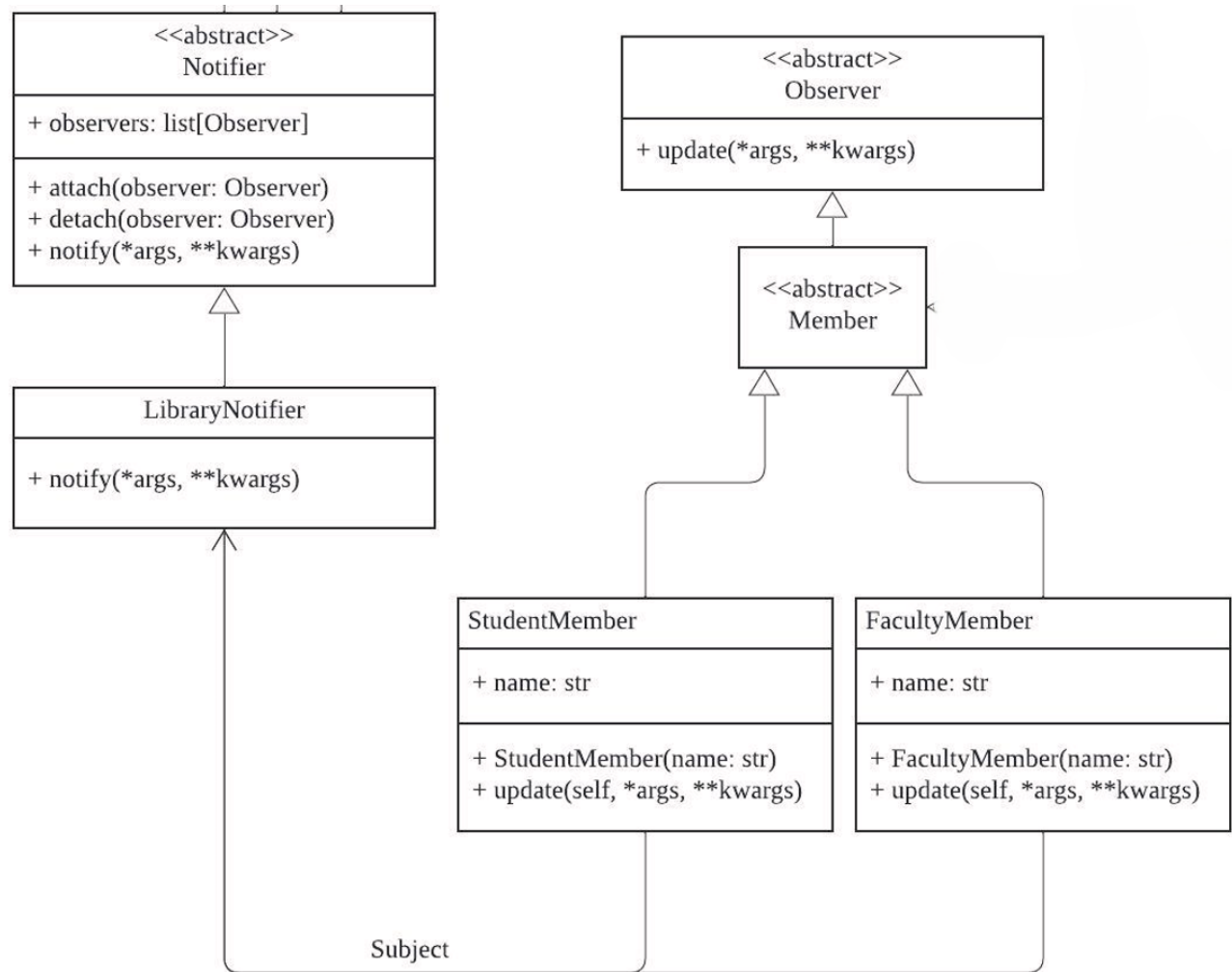
- **Classes:** **Command**, **BorrowBookWithAccessoriesCommand**, **ReturnBookCommand**, **AddBookCommand**, **AddMemberCommand**, **SearchBooksCommand**, **DisplayBorrowedBooksCommand**, **AddThirdPartyBooksCommand**, **DisplayAvailableBooksCommand**, **DisplayUsersCommand**
- **Purpose:** Encapsulate a request as an object, thereby allowing users to parameterize clients with queues, requests, and operations, as well as decoupling the Client/UI from detailed implementation.
- **Implementation:** The **Command** abstract class defines the `execute` method, which is implemented by various command classes to perform operations like borrowing books, returning books, adding members, and displaying information.

Decorator Pattern



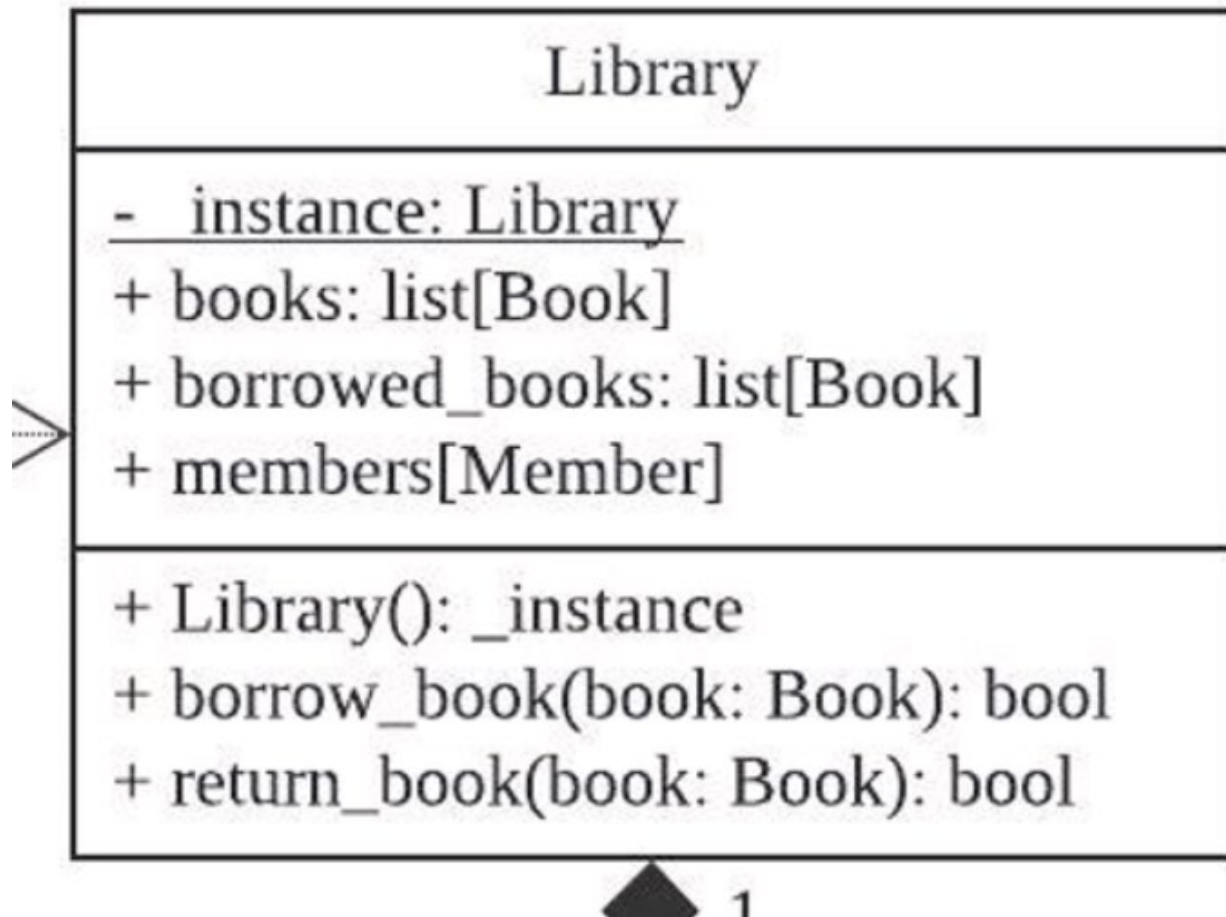
- **Classes:** **Book**, **BookDecorator**, **BookAccessoryDecorator**
- **Purpose:** Allow behavior to be added to individual objects dynamically and locally, without affecting the behavior of other objects of same class.
 - Eg. **BookAccessoryDecorator** has no effect to the Ebook/PrintBook, and every time `get_description()` is called, it stacks its value together with the value of `get_description()` generated from its child `decoratedBook`.
- **Implementation:** The **BookAccessoryDecorator** class extends the **BookDecorator** class and adds additional functionality (e.g., accessories) to the book objects.

Observer Pattern



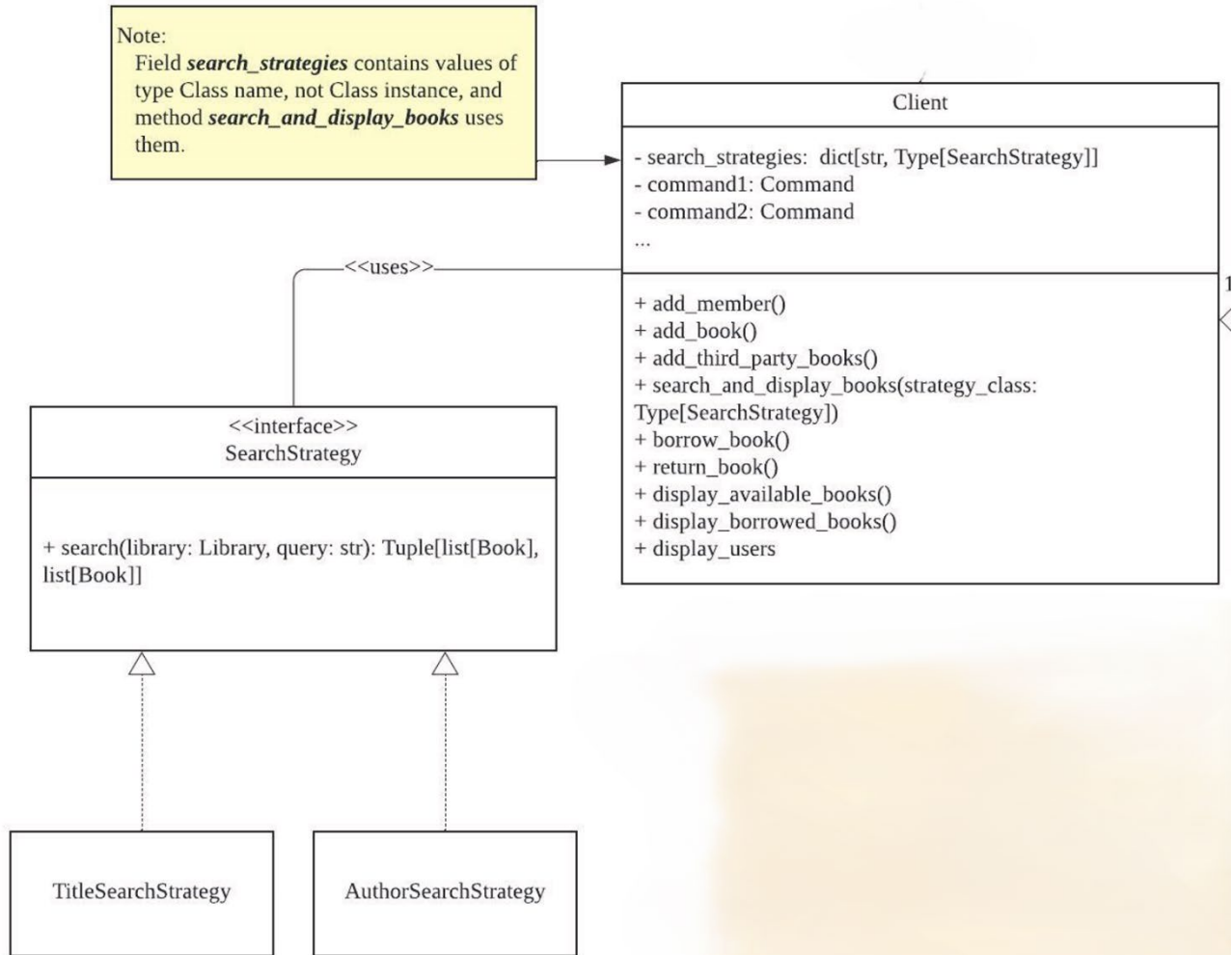
- **Classes:** **Notifier**, **LibraryNotifier**, **Member**, **StudentMember**, **FacultyMember**
- **Purpose:** Establish a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Implementation:** The **Notifier** class maintains a list of observers (e.g., **Member** objects) and notifies them when a new book is added. **LibraryNotifier** extends **Notifier** and provides specific notification functionality.

Singleton pattern



- **Class:** **Library**, **LibraryNotifier**, **MemberFactory**
- **Purpose:** Ensure that there is only one instance of these classes throughout the application to manage books, members, and notifications consistently.
- **Implementation:**
 - Each of these classes have a private static instance and a `__new__` method that ensures only one instance is created.

Strategy Pattern



- **Classes:** **SearchStrategy**, **TitleSearchStrategy**, **AuthorSearchStrategy**
- **Purpose:** Define a family of algorithms, encapsulate each one, and make them interchangeable. The strategy pattern allows the algorithm to vary independently from clients that use it.
- **Implementation:** The `SearchStrategy` abstract class defines the abstract search method, which is implemented by `TitleSearchStrategy` and `AuthorSearchStrategy` to provide different search criteria.